

Notebook Construction and Audit Strategies

1 This section defines two complementary frameworks for computational notebooks in machine-
 2 learning and data-science contexts. The *Construction Framework* governs the creation and revi-
 3 sion of notebook artifacts, whereas the *Audit Framework* provides an independent, criteria-based
 4 evaluation of notebooks that have reached an auditable level of completeness and stability.

NOTEBOOK DEVELOPMENT AND ASSURANCE LIFECYCLE

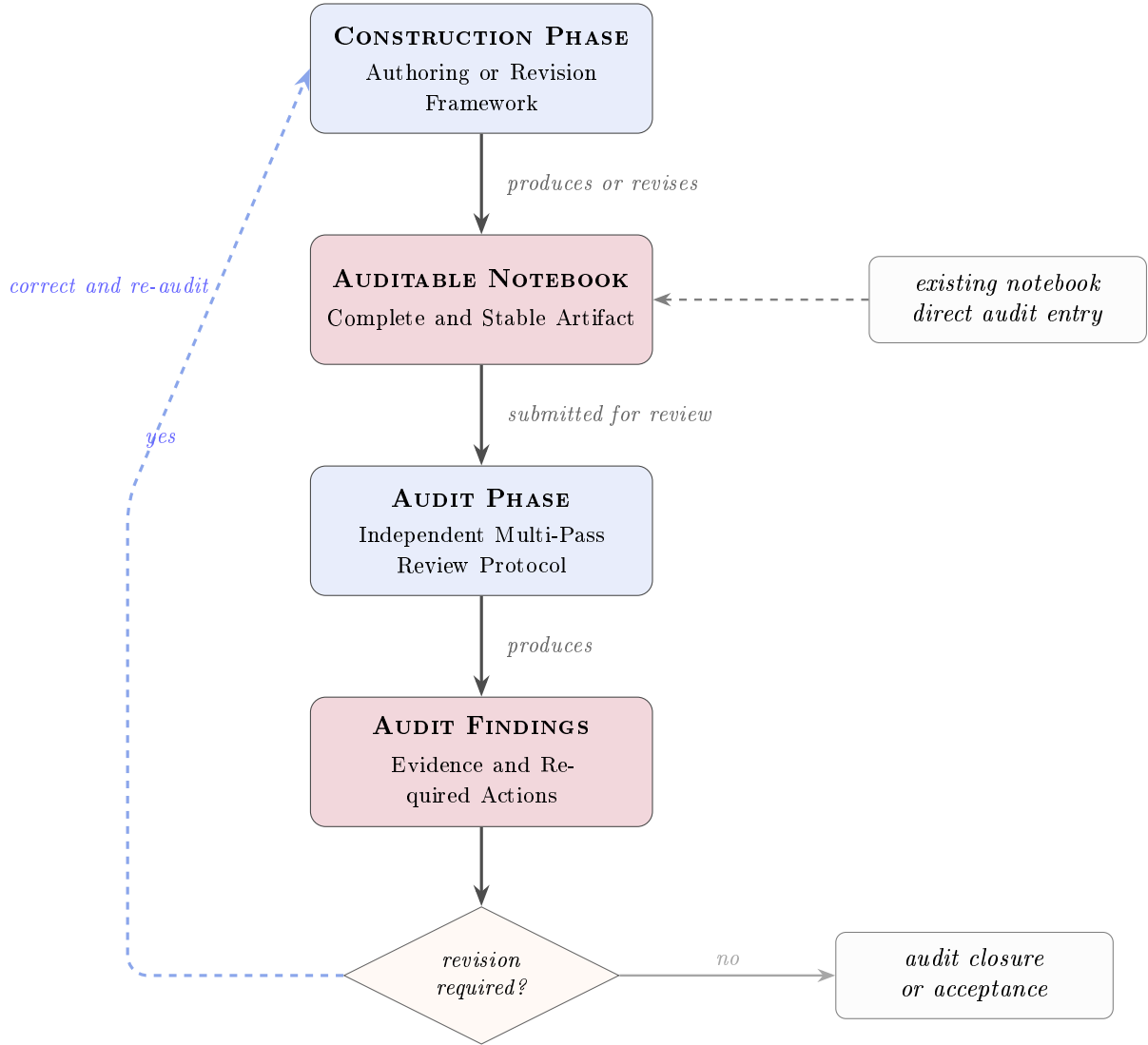


Figure 1: The Construction Framework creates or revises auditable notebooks, while the Audit Framework independently evaluates them. New notebooks typically enter through construction; complete existing notebooks may enter directly at audit. Audit findings determine whether the notebook proceeds to closure or returns to construction for revision and re-audit.

5 The relationship between the frameworks is therefore staged but not universally linear: newly
 6 authored notebooks normally progress from construction to audit, while existing notebooks may
 7 enter directly into the audit process. In either case, the audit outcome determines the subsequent
 8 path. A notebook for which no revision is required proceeds to audit closure, whereas required
 9 corrections are implemented through the Construction Framework and subsequently verified by
 10 re-audit. This conditional lifecycle is illustrated in Figure 1.

Part I Notebook Construction Framework

The Construction Framework is a *forward-looking prescription*: it guides the notebook author in producing a well-structured, reproducible, and maintainable artifact. It comprises three coordinated phases: scaffolding, incremental writing, and validation during development and before release.

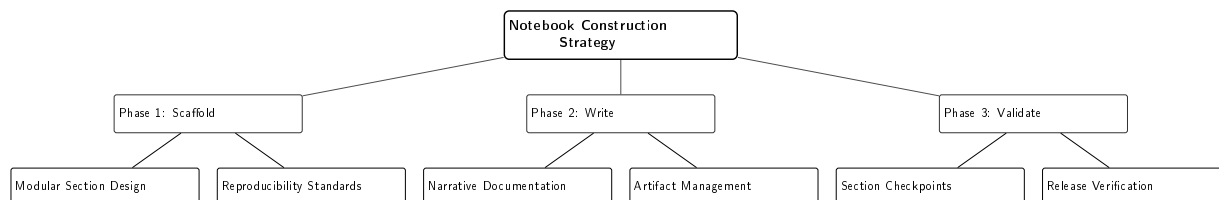


Figure 2: High-level pillars of the Notebook Construction Framework.

Phase 1 Scaffold: Establish the structural, data-identity, and reproducibility foundation before implementing downstream preprocessing, modeling, or evaluation logic:

- Define the notebook’s **single responsibility**: one notebook should correspond to one coherent workflow or experiment.
- Create standard section headers as markdown cells, following this canonical order:
 1. Environment & Dependencies
 2. Configuration & Global Parameters
 3. Data Ingestion & Schema Validation
 4. Data Identity, Partitioning & Split Manifest
 5. Preprocessing & Feature Engineering
 6. Model Definition & Training
 7. Evaluation & Metrics
 8. Artifact Export
 9. Conclusions & Next Steps

Data Identity, Partitioning, and Split-Manifest Contract When data partitioning is applicable, the *Data Identity, Partitioning & Split Manifest* section shall define the valid observation set, assignment unit, split strategy, split keys, random or chronological boundaries, grouping constraints, stratification rules, filtering and deduplication rules, and partition assignments before fitted preprocessing, feature selection, model fitting, threshold selection, hyperparameter selection, or model selection is performed.

Deterministic schema validation, filtering, deduplication, and identifier extraction required to establish the valid observation set may precede partition assignment, provided that they do not use fitted parameters, evaluation outcomes, or downstream information.

The Split Manifest shall be bound to the exact source-data snapshot from which the partition assignment was generated. At minimum, it shall record:

- the dataset name and immutable version or equivalent snapshot reference;
- the applicable schema or data-contract version;
- the expected observation or group identifier set, represented directly or through a versioned artifact with an integrity reference;
- the stable identifier and assignment unit used for partitioning;
- the partition assigned to each observation or group;
- the partitioning method, configuration, seeds, boundaries, grouping keys, stratification variables, filtering rules, exclusion criteria, and applicable tolerances; and
- the integrity reference for the completed Split Manifest and any linked assignment artifact.

Observations shall be assigned directly when no grouping dependency applies. When grouping constraints apply, all observations governed by the same group identifier shall inherit the same partition.

A split assignment shall remain valid only for the dataset snapshot, schema, observation set, assignment unit, filtering and deduplication rules, grouping policy, and partitioning configuration recorded in the Split Manifest. Any change that can alter dataset membership, partition assignment, dependency preservation, or leakage risk shall invalidate the manifest and require generation of a new one before the split is consumed.

A split is consumed when partition membership is used to fit, transform, select, tune, train, evaluate, compare, or report results.

Before consumption, Phase 3 verification shall confirm that the dataset identity and schema match the Split Manifest, that each required observation or group has exactly one valid assignment, that no prohibited overlap exists, that all declared constraints are satisfied within the recorded tolerances, and that the manifest and assignment-artifact integrity references are valid.

Phase 2 Write: Author each section incrementally, following these discipline rules:

- Work through **one section at a time** (Divide and Conquer principle).
- Provide markdown explanations for each logical code block, stating its intent, declared inputs, and expected outputs. Adjacent cells implementing one operation may share an explanation.
- Avoid hiding material logic inside loops or helper functions without documenting their purpose, assumptions, and side effects.
- Prefer **explicit variable passing** over hidden dependencies or mutable global state.
- Summarize repetitive patterns and document their meaningful variations. Three or more substantially similar blocks shall be treated as a refactoring candidate. Extracted logic may be moved to a local module only when it is versioned, included in the project repository, referenced by the notebook or project configuration, and reviewed with the notebook.
- Route all persisted outputs, including models, plots, and reports, through the versioned export convention declared in the notebook configuration. Persisted artifacts shall not be written ad hoc from arbitrary cells.

Phase 3 Validate During Writing and Before Release: Verify correctness at defined checkpoints during Phase 2 and perform end-to-end verification before release, while maintaining strict separation between development executions and release-bearing runs.

Notebook-Execution-Attempt and Release-Run Contract Every checkpoint, exploratory, partial, or complete execution shall receive a unique *Notebook Execution Attempt Identifier* (NEAI). Its execution record shall identify:

- the governing notebook, environment, configuration, data, and manifest versions;
- the notebook section or workflow stage executed;
- the execution status;
- the artifacts produced; and
- whether the execution was partial or complete.

Artifacts produced by checkpoint, exploratory, partial, failed, or interrupted executions shall be stored in a development namespace associated with the NEAI. They shall not be promoted into, interpreted as, or used to verify release-run artifacts.

A complete execution intended for release shall begin from a fresh kernel under a new NEAI and an isolated provisional namespace. A unique release-bearing *Run Identifier* shall be assigned only after the complete workflow finishes successfully. Failed or interrupted executions shall remain execution attempts and shall not acquire a release-run state.

A successfully completed release run shall enter the following states:

COMPLETE_UNVERIFIED and COMPLETE_VERIFIED.

The run shall remain COMPLETE_UNVERIFIED until all required section contracts, data constraints, expected outputs, and artifact-integrity checks have been verified. Only then may it transition to COMPLETE_VERIFIED and support release, publication, or deployment.

- At each section checkpoint, apply the *Notebook-Execution-Attempt and Release-Run Contract* above and verify the section’s declared inputs, outputs, side effects, execution order, and applicable deterministic checks or stochastic tolerances.
- Before partition membership is consumed, complete the formal Split Manifest verification defined in Phase 1.
- Resolve failed checkpoint controls before proceeding to the next section.
- After all applicable sections are complete, apply the release-run procedure and state transitions defined exclusively by the *Notebook-Execution-Attempt and Release-Run Contract* above.

Part II Notebook Audit Framework

The Audit Framework is a *backward-looking diagnostic*: it guides a reviewer, collaborator, or AI assistant in systematically assessing the quality, correctness, and risk profile of an existing notebook.

Audit Package Boundary The audit documentation shall use the following manifest hierarchy:

- the *Audit Package Manifest* establishes the immutable identity of the supplied audit package using the required contents defined below;

- the *Audit Scope Manifest* records the applicability and scope classification of each audit step;
- the *Audit Manifest* is the cumulative working record of execution evidence, provisional findings, verification outcomes, and gate states; and
- the *Final Audit Manifest* is the verified, finalized form of the Audit Manifest containing the final findings, severities, risk score, gate decisions, and audit status.

Before execution, the Audit Package Manifest shall enumerate all supplied files, their project-relative paths, and their cryptographic hashes. The audit target shall comprise the complete executable project surface required by the notebook:

- the `.ipynb` file;
- every transitively imported project-local module;
- project configuration and schema files;
- environment and dependency manifests;
- referenced local data contracts or split manifests; and
- scripts used to execute, export, or validate the notebook.

Third-party package source code is outside the audit boundary unless explicitly included, but its package name and resolved version remain part of the reproducibility review.

If required project-local code or configuration is absent, a Critical reproducibility finding shall be recorded. Methodological or deployment logic delegated to a supplied local module shall be audited as equivalent inline notebook logic.

To execute this efficiently, the reviewer should divide the notebook into logical modules, prioritize critical pipeline blocks over repetitive exploration cells, use hierarchical descriptions, and summarize repeated code patterns. Before execution, every audit step shall be classified as `IN_SCOPE`, `OUT_OF_SCOPE`, or `NOT_APPLICABLE`.

A user-requested exclusion shall produce `OUT_OF_SCOPE` only when the governing control expressly permits exclusion. A control declared mandatory or non-excludable shall remain `IN_SCOPE`; a request to exclude it shall be rejected and recorded in the Audit Scope Manifest without changing its scope classification. The mandatory Pass 1 safety pre-screen is non-excludable and shall never be classified as `OUT_OF_SCOPE` or `NOT_APPLICABLE`.

A step shall be classified as `NOT_APPLICABLE` only when the notebook type does not contain the corresponding operation and the governing control does not declare the step universally mandatory. The reviewer shall record objective evidence for every `NOT_APPLICABLE` determination.

Audit Control and Step Identity Contract An *Audit Control* is one top-level `\item` within a Pass 1–6 `itemize` environment, including an assessment criterion, deliverable requirement, deterministic classification rule, gate rule, or invalidation rule. Controls shall be identified in physical top-to-bottom order within the versioned framework snapshot using the format `P{pass}-C{ordinal}`, for example `P2-C04`. The ordinal shall be zero-padded to two digits and shall include every top-level item in the applicable pass.

The framework publisher shall decompose each Audit Control into an ordered, complete list of independently truth-valued predicates in the versioned *Defect Assertion Registry*. Each registry entry shall contain exactly one predicate identifier, the exact contiguous source span from which the predicate is derived, and exactly one declarative Boolean assertion whose truth value can be

165 assigned independently of every other registry entry. Predicate identifiers shall follow physical
166 registry order using `P{pass}-C{control-ordinal}-Q{predicate-ordinal}`. The ordered reg-
167 istry entries, rather than implementation-time parsing of prose, shall be the exclusive and final
168 predicate decomposition for the framework snapshot. Every normative requirement, prohibition,
169 classification condition, gate condition, and invalidation condition in an Audit Control shall be
170 represented by exactly one registry entry; two entries shall not contain the same Boolean assertion.
171 Before any Audit Step executes, the Audit Scope Manifest shall contain this versioned Defect As-
172 sertion Registry and shall map every predicate identifier to exactly one canonical Defect Assertion
173 Code.

174 Each canonical Defect Assertion Code shall have exactly one immutable *Defect Signature* consist-
175 ing, in order, of the controlled subject type, normalized governed-property identifier, prohibited
176 or required relation, and normalized comparison value. Two predicates across passes shall share a
177 Defect Assertion Code if and only if all four Defect Signature fields are byte-for-byte identical after
178 the registry's declared Unicode NFC normalization. The registry shall enumerate the complete
179 permitted value set for each Defect Signature field; field values not present in those versioned
180 sets are invalid. An explicit alias entry shall cite the two predicate identifiers, their identical
181 Defect Signature, and the shared code. Predicates whose signatures differ in any field shall use
182 different codes. Codes, signatures, and alias entries shall remain immutable within the framework
183 snapshot.

184 An *Audit Step* is the execution record of exactly one Audit Control against one audited source
185 snapshot. Its identifier shall use the format `P{pass}-C{ordinal}@{source-snapshot-hash}`. An
186 Audit Step shall not combine multiple Audit Controls, and one Audit Control shall not be split
187 into multiple Audit Steps. When a control contains multiple predicates, all of its mandatory
188 predicates shall be verified before the corresponding Audit Step may transition to `COMPLETED`;
189 predicate-level evidence shall not create additional Audit Steps.

190 Before any Audit Step is executed, the Audit Scope Manifest shall enumerate the complete Audit
191 Control registry, the corresponding Audit Step identifiers, their pass membership, scope classifi-
192 cations, and required deliverables. It shall also contain a complete ordered *Sequential LLM Level*
193 *Registry* for Passes 1–6. Each registry entry shall contain a unique LLM-Level Identifier, pass
194 membership, the complete set of covered Audit Step identifiers, sequential ordinal, and initial
195 LLM-Level Status `PENDING`. The registry shall contain every sequential-protocol LLM level that
196 may execute or receive `NOT_EXECUTED_GATED`; no registered level shall cover only `OUT_OF_SCOPE`
197 or `NOT_APPLICABLE` Audit Steps. Sequential-protocol LLM levels shall not be created, deleted,
198 merged, split, or have their covered Audit Steps changed after the first Audit Step begins. Sup-
199plementary LLM levels are not sequential-protocol LLM levels and shall be registered only under
200 a Supplementary Review Authorization. References to a *control* shall denote the versioned nor-
201 mative rule; references to an *Audit Step* shall denote its snapshot-bound execution record. LLM
202 levels, findings, evidence records, gate records, and verification records shall cite the applicable
203 Audit Step identifiers. Implementation- defined merging, splitting, renumbering, or omission of
204 controls is prohibited within a framework snapshot.

205 Passes 2–6 shall execute every `IN_SCOPE` step unless a deterministic gate explicitly terminates
206 the remaining protocol. The status of each audit step shall be recorded separately from the
207 status of its containing LLM level. A gated step shall be recorded as Audit-Step Execution
208 Status `NOT_EXECUTED_GATED` and shall not be treated as completed. Neither `OUT_OF_SCOPE` nor
209 `NOT_APPLICABLE` contributes to the risk score. An unrestricted audit requires completion of every
210 applicable step with no user exclusions or gated omissions.

211 The framework is formalized as a six-pass review protocol, applied sequentially. Any pass included
212 in scope is executed according to its applicability classification and deliverable requirements below.

Finding Severity and Deterministic Risk Assignment Every finding shall be assigned exactly one severity:

Critical: The finding prevents required execution, invalidates scientific results, exposes credentials or sensitive data, or independently blocks release, deployment, or publication.

Major: The finding materially weakens reproducibility, methodological correctness, maintainability, or deployment readiness but is not independently release-blocking.

Minor: The finding affects clarity, documentation, output hygiene, or non-blocking maintainability without changing execution or results.

The aggregate audit risk score is determined from all verified findings as follows:

- *Low Risk:* no Critical findings, no Major findings, and no more than two Minor findings.
- *Moderate Risk:* no Critical findings and either one or two Major findings or three or more Minor findings.
- *High Risk:* one or more Critical findings overall, or three or more Major findings across all passes.

Across the complete audit, three or more verified Major findings shall escalate the aggregate audit risk to High Risk, even when no single pass contains three Major findings. A finding shall not affect a final risk score until its evidence and source reference have been verified. Generated findings and pass claims remain provisional until verification is complete.

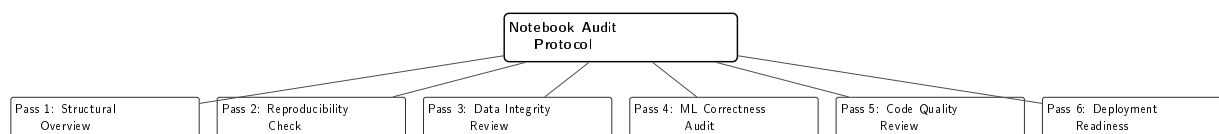


Figure 3: High-level pillars of the Notebook Audit Framework.

Pass 1 Structural Overview: Establish a high-level map of the notebook before examining any logic:

- Record any user-specified focus areas or exclusions that will scope the remaining five passes.
- Assess whether the notebook adheres to the canonical section headers defined in the Construction Framework (e.g., Environment, Configuration, Ingestion).
- Confirm that the notebook has a single, coherent purpose.
- Verify that cell execution order is linear and safe.
- Identify execution-relevant structural red flags (e.g., broken imports, missing required cells, unsafe execution order, or unresolved project-local dependencies). Code smells and unused elements remain exclusive to Pass 5.
- Perform the mandatory safety pre-screen: scan the notebook for exposed credentials, API keys, personally identifiable information (PII), and potentially destructive shell or system commands. This pre-screen cannot be excluded and must complete before any other stop condition is evaluated. Each identified safety risk shall be assigned exactly one Safety-Risk Disposition: `REJECTED_AS_NOT_CREDIBLE`, `CONTAINED`, `EXPLICITLY_AUTHORIZED`, or `UNRESOLVED`. If any safety risk remains `UNRESOLVED`, the verified pre-screen shall emit Gate Event `UNRESOLVED_SAFETY_RISK` under the *Gate*

State Transition Contract. After the resulting transition to **STOP**, Passes 2–6 shall be recorded as Audit-Step Execution Status **NOT_EXECUTED_GATED**, and the Audit Status shall be **AUDIT_INCOMPLETE**. If no safety risk remains **UNRESOLVED**, the remaining Pass 1 controls may proceed; completion of the safety pre-screen alone shall not emit **PASS1_VERIFIED_COMPLETE**.

- *Deliverable:* Section map, structural findings, mandatory safety pre-screen, and Audit Scope Manifest.

Pass 2 Reproducibility Check: Assess whether the notebook can be reliably re-executed:

- Verify that dependencies are declared and version-pinned.
- Confirm that stochastic controls are configured for the tech stack (e.g., global random seeds and framework-specific deterministic flags for deep learning workloads).
- Flag hardcoded non-secret file paths and environment-specific assumptions that impair portability. Credential exposure is governed exclusively by the mandatory Pass 1 safety pre-screen and shall not be recorded again as a separate Pass 2 finding.
- After the mandatory Pass 1 safety pre-screen has completed and no unresolved safety block remains, execute the notebook end-to-end from a fresh kernel in an isolated environment constructed from the declared dependency manifest. Record the command, environment hash, exit status, execution log, runtime, and produced artifact manifest. A verified Low or Moderate reproducibility classification shall not be issued without this execution evidence.

- *Deterministic Pass-Level Risk Assignment:*

After de-duplication, let C_2 , M_2 , and m_2 denote, respectively, the numbers of distinct verified Critical, Major, and Minor findings attributable to the structural, dependency, execution, and safety controls assessed for Pass 2. Each distinct finding shall be counted once in the Pass 2 classification, regardless of the number of controls or source references that identify it.

The Pass 2 reproducibility risk classification shall be assigned as follows:

- *Low Risk:* $C_2 = 0$, $M_2 = 0$, and $m_2 \leq 2$;
- *Moderate Risk:* $C_2 = 0$ and either $1 \leq M_2 \leq 2$ or $m_2 \geq 3$; and
- *High Risk:* $C_2 \geq 1$ or $M_2 \geq 3$.
- *Deliverable:* Clean-execution record and deterministic Pass 2 reproducibility risk classification (*Low Risk*, *Moderate Risk*, or *High Risk*). The classification shall record C_2 , M_2 , m_2 , and the verified finding identifiers included in those counts. This pass-level classification contributes to, but does not replace, the aggregate audit risk score determined from all verified findings.
- *Deterministic Gate:* A *High Risk* Pass 2 reproducibility classification shall emit Gate Event **DETERMINISTIC_STOP_VERIFIED** with Stop Reason **HIGH_RISK_REPRODUCIBILITY** under the *Gate State Transition Contract*. Subsequent step disposition and any prospective resumption shall be governed exclusively by *Protocol Overrides and Supplementary Reviews*. An audit that remains terminated by this gate shall receive an Audit Status of **AUDIT_INCOMPLETE** and shall not receive an unrestricted audit status.

Pass 3 Data Integrity Review: Examine the data pipeline for correctness and leakage risks:

292 • Confirm that no information from the test or validation set leaks into the training
293 process (e.g., ensure imputers, scalers, encoders, or feature-selection algorithms are
294 fitted only on the training data).

295 • Verify that missing data is handled consistently across splits.

296 • Check schema, data types, shapes, ranges, and invariant constraints at ingestion. Dis-
297 tributational profiling used to guide model or preprocessing decisions shall be restricted
298 to the training partition; test-partition checks shall be limited to predefined contract
299 validation and final evaluation.

300 • *Deliverable:* Data pipeline integrity report.

301 • *Deterministic Performance-Claim Invalidation Rule:*

302 Verified pipeline data leakage shall invalidate every affected model-performance value,
303 comparison, selection decision, and conclusion. Each affected claim shall receive a
304 Performance-Claim Validity Status of **INVALIDATED**, and the associated finding shall
305 receive a severity of Critical.

306 This invalidation rule does not independently change the Gate State. Pass 4 method-
307 ological design checks shall remain executable and reportable using separate checklist
308 statuses of **SATISFIED** or **NOT_SATISFIED**. An invalidated performance claim shall not
309 be reported as valid solely because an associated Pass 4 design criterion is **SATISFIED**.

310 **Pass 4 ML Correctness Audit:** Evaluate the soundness of the machine-learning methodology:

311 • Confirm that the evaluation metric is appropriate for the task and class distribution.
312 Verify that cross-validation is applied correctly and without leakage.

313 • Ensure hyperparameter selection uses only training-derived resampling, cross-
314 validation, or a designated validation partition. Nested cross-validation is valid when
315 the outer assessment folds remain isolated. The final test set shall never guide tuning
316 or model selection.

317 • Check that model performance is contextualized against a meaningful baseline.

318 • *Deliverable:* ML correctness checklist with each applicable criterion marked **SATISFIED**
319 or **NOT_SATISFIED**, accompanied by severity-classified findings and a separate
320 performance-claim validity status when leakage is verified.

321 **Pass 5 Code Quality Review:** Assess the maintainability and clarity of the code:

322 • Flag three or more substantially similar code blocks as refactoring candidates.

323 • Identify dead code, unused imports, and redundant variable assignments.

324 • Assess whether variable and function names are meaningful and consistent.

325 • Check for bloated cell outputs (e.g., printing entire dataframes or raw tensors).

326 • *Deliverable:* Code quality and smell report whose verified findings contribute to the
327 aggregate risk score.

328 **Pass 6 Deployment Readiness:** Determine whether the notebook is suitable for production or
329 publication:

330 • Audit compliance with the *Notebook-Execution-Attempt and Release-Run Contract*
331 defined in Part I, including notebook-execution-attempt identity, namespace separa-

tion, release-run eligibility, verification-state transitions, artifact immutability, and re-execution identity. Record each violated contract provision as a separate finding.

- Confirm that inference logic is separable from training logic.
- Check that compute and memory constraints are documented or profiled.
- Ensure a complete environment export is present and up to date.
- Assess deployment-specific privacy and PII handling controls using the Pass 1 safety findings as inputs. A Pass 6 finding shall be recorded only for a distinct deployment control failure not already represented by a Pass 1 finding.
- *Deterministic Pass-Level Risk Assignment:*

After de-duplication, let C_6 , M_6 , and m_6 denote, respectively, the numbers of distinct verified Critical, Major, and Minor findings attributable to the deployment controls assessed in Pass 6. Each distinct finding shall be counted once in the Pass 6 classification, regardless of the number of controls or source references that identify it.

The Pass 6 deployment-readiness risk classification shall be assigned as follows:

- *Low Risk*: $C_6 = 0$, $M_6 = 0$, and $m_6 \leq 2$;
- *Moderate Risk*: $C_6 = 0$ and either $1 \leq M_6 \leq 2$ or $m_6 \geq 3$; and
- *High Risk*: $C_6 \geq 1$ or $M_6 \geq 3$.
- *Deliverable*: Deterministic Pass 6 deployment-readiness risk classification (*Low Risk*, *Moderate Risk*, or *High Risk*). The classification shall record C_6 , M_6 , m_6 , and the verified finding identifiers included in those counts. This pass-level classification contributes to, but does not replace, the aggregate audit risk score.

Applying Both Frameworks

Framework selection depends on the notebook's development state and the purpose of the engagement:

- *Construction*: new notebook or experiment authoring, reusable-template development, or revision in response to established requirements or audit findings.
- *Audit*: formal review of a sufficiently complete notebook for deployment, publication, pre-merge approval, or assessment without modification.
- *Combined lifecycle*: construction followed by audit, with subsequent activity governed by the conditional paths in Figure 1.

LLM Execution of Construction and Audit Strategies

The use of a Large Language Model (LLM) within the Construction and Audit Frameworks requires reproducible analysis and evidence-based review. The general limitations below apply to both frameworks; the status, gate, runner, verification, and adjudication controls apply specifically to formal audits.

- **Triage-Sensor Role**: An LLM may identify structural patterns and potential defects, but shall not be assumed to trace complex dynamic state, indirect dependencies, or deeply nested logic reliably.

- **Context-Window Constraints:** A chained, multi-step architecture may reduce, but does not eliminate, context degradation. When the notebook exceeds the effective capacity of the selected model and prompt configuration, it shall be divided into logical, traceable chunks. Tool-assisted execution and state inspection may supplement this analysis, provided that the notebook and its dependencies remain fully covered.
- **Permitted Scope:** During construction, the LLM may assist with scaffolding, documentation, code review, and checkpoint analysis. During formal audit, it may generate the provisional outputs permitted by the authority boundary below.

Authority Boundary and Status Model

An audit pass defines a review domain, while an LLM level defines a bounded analysis stage within a pass. A pass may contain one or more LLM levels. Each sequential-protocol level shall be defined before execution in the Sequential LLM Level Registry and shall identify the pass and complete set of Audit Steps that it covers. A supplementary LLM level shall instead be defined by its Supplementary Review Authorization.

The formal analysis hierarchy shall be:

Audit Pass ($\supset$) LLM Level ($\supset$) Analysis Attempt ($\supset$) Lap.

An *Analysis Attempt* is one invocation sequence used to execute an LLM level and shall receive a unique *Analysis Attempt Identifier (AAI)*. A *Lap* is the ordinal label of one prompt invocation within an AAI, beginning with LAP-01. A Lap is not an Audit Pass, an LLM Level, an Audit Step, or a status-bearing governance object. Every Lap record shall identify its AAI, containing LLM Level, Audit Pass, covered Audit Step identifiers, and ordinal position.

Gating mode and *non-gating mode* are Lap execution-mode attributes. Gating mode emits the provisional workflow metadata required by the Verification Workflow; non-gating mode emits extraction content without embedded gate metadata. Neither mode changes the governing LLM-Level Status, Gate State, verification, or adjudication rules.

Extraction-Record and Provisional-Finding Promotion Contract The descriptive objects `AtomicObservation`, `ComplexFunction`, and `CrossReferenceDiscrepancy` are *Extraction Records*; they are not findings and shall not independently contribute to severity, scoring, gate decisions, or compliance claims. Each Extraction Record shall receive a stable identifier composed of its AAI, Lap ordinal, record type, and record ordinal.

An Extraction Record may enter adjudication only through a distinct promotion stage after extraction. A promotion record shall identify one or more source Extraction Record identifiers, one or more affected Audit Step identifiers, the source snapshot hash, evidence references, affected artifacts, stable source references, affected-object identity, one or more Defect Assertion Codes from the versioned Defect Assertion Registry, and descriptive finding text. The promotion stage shall compute the canonical Finding Key defined below for each Extraction Record. Extraction Records with identical Finding Keys shall produce one provisional finding; Extraction Records with different Finding Keys shall produce separate provisional findings. An Extraction Record may support at most one provisional finding, while one provisional finding may cite multiple Extraction Records and Audit Steps.

Each promoted object shall receive a stable Provisional Finding Identifier and a Finding Adjudication Status of `PROVISIONAL`. The promotion may be generated by an LLM level but shall remain provisional until verified under the Verification Workflow. Extraction Records that are not promoted shall remain descriptive audit evidence and shall not enter finding de-duplication or adjudication.

415 **Finding Identity and De-duplication Contract** Every provisional finding, regardless of
416 whether it originates from an Extraction Record, a human reviewer, or an approved deterministic
417 tool, shall receive exactly one canonical Finding Key. The pre-hash Finding Key tuple shall
418 contain, in this order:

- 419 1. the uppercase hexadecimal audited source-snapshot hash;
- 420 2. the sorted, duplicate-free list of normalized affected project-relative artifact paths, or the
421 literal `NONE`;
- 422 3. the sorted, duplicate-free list of Stable Source-Reference Protocol references, or the literal
423 `NONE`;
- 424 4. the sorted, duplicate-free list of Defect Assertion Codes from the versioned Defect Assertion
425 Registry;
- 426 5. the affected-object type; and
- 427 6. the stable affected-object identifier.

428 Project-relative paths shall use forward slashes, shall contain no `.` or `..` segments, and shall
429 preserve case. Stable source references shall use the exact canonical representation defined by the
430 Stable Source-Reference Protocol. List values shall be sorted by ascending Unicode code-point
431 order after normalization.

432 Affected-object type shall be exactly one of `NOTEBOOK`, `CELL`, `MODULE`, `DATASET`, `PARTITION`, `MODEL`,
433 `ARTIFACT`, `ENVIRONMENT`, `EXECUTION`, `RELEASE`, `OTHER`, or `NONE`. For every type other than `NONE`, the
434 affected-object identifier shall be the canonical identifier recorded in the Audit Package Manifest,
435 Split Manifest, execution record, or artifact manifest and shall be encoded after Unicode NFC
436 normalization without case folding or other substitution. Type `OTHER` shall use the identifier
437 format `{namespace}:{canonical-identifier}`, where the namespace is registered in the Audit
438 Package Manifest. Type `NONE` shall use the identifier `NONE` and is valid only when no governed
439 object is affected.

440 The Audit Manifest shall record the Unicode Standard version used for NFC and code-point
441 ordering. That version shall remain immutable for the audit, and every implementation that
442 computes or verifies a Finding Key shall use the recorded version.

443 The pre-hash tuple shall use the fixed field tags `SNAPSHOT`, `ARTIFACTS`, `REFERENCES`,
444 `ASSERTION_CODES`, `OBJECT_TYPE`, and `OBJECT_ID`, in that order, preceded by the ASCII version
445 marker `FK1;`.

446 A scalar field shall be serialized as its ASCII tag, the ASCII string `=S`, the canonical unsigned-
447 decimal UTF-8 byte length, one colon, the exact field bytes, and one semicolon. A list field
448 shall be serialized as its ASCII tag, the ASCII string `=L`, the canonical unsigned-decimal member
449 count, one colon, each member serialized as its canonical unsigned-decimal UTF-8 byte length,
450 one colon, and its exact bytes, followed by one semicolon after the final member. A canonical
451 unsigned-decimal integer shall contain only ASCII digits `0–9`, shall contain no sign or surrounding
452 whitespace, and shall contain no leading zero unless its value is zero, which shall be encoded
453 exactly as `0`. The member count and each member length shall be computed after normalization.
454 Empty lists are prohibited; the single-member list containing literal `NONE` shall represent no value
455 for `ARTIFACTS` or `REFERENCES`.

456 The Finding Key shall be the uppercase 64-character hexadecimal SHA-256 digest of the complete
457 tagged serialization. The Provisional Finding Identifier shall be `PF-{Finding-Key}`.

Two provisional findings shall be duplicates if and only if every component of their pre-hash Finding Key tuples is identical and therefore their Finding Keys are identical. Audit Step identifiers, evidence source, reporting pass, LLM level, Lap, recommended severity, causal explanation, and descriptive text are finding metadata and shall not alter the Finding Key. Findings with different Finding Keys shall remain distinct.

When duplicates exist, their shared PF-**{Finding-Key}** shall be the single surviving identifier; all duplicate records shall reference it. Evidence, Audit Step identifiers, source references, causal explanations, and severity recommendations shall be aggregated under the surviving identifier without multiplying the finding count. Pass-level and aggregate risk calculations shall count distinct **CONFIRMED** Finding Keys only.

An LLM may generate provisional findings, severity recommendations, applicability classifications, checklist statuses, and gate recommendations. It shall not independently issue a final **PASS**, **FAILED**, Gate State **STOP**, **INVALIDATED** determination, risk score, or release decision. The absence of an LLM-generated finding shall not constitute evidence of compliance.

All executed and gated LLM levels shall follow the status transitions and verification authority defined exclusively in the **Verification Workflow**. Until that workflow reaches a terminal verification state, every LLM-generated outcome shall remain provisional.

The workflow shall distinguish:

- *Audit-Step Scope Classification*: **IN_SCOPE**, **OUT_OF_SCOPE**, or **NOT_APPLICABLE**. This classification shall remain immutable within an audit snapshot after scope verification;
- *Audit-Step Execution Status*: **PENDING**, **PENDING_COMPLIANT_REEXECUTION**, **COMPLETED**, or **NOT_EXECUTED_GATED**. This status shall change independently of the Audit-Step Scope Classification;
- *LLM-Level Status*: **PENDING**, **PENDING_COMPLIANT_REEXECUTION**, **PAUSED_FOR_VERIFICATION**, **VERIFIED**, or **NOT_EXECUTED_GATED**;
- *Gate State*: **INITIAL**, **PAUSE**, **PROCEED**, or **STOP**, governed exclusively by the **Gate State Transition Contract**;
- *Gate Transition Violation Status*: **PENDING_ADJUDICATION**, **REJECTED_NO_EFFECT**, **CORRECTED_BY_VALID_EVENT**, or **UNRESOLVED**, governed exclusively by the **Gate State Transition Contract**;
- *Supplementary Review Status*: **AUTHORIZED**, **EXECUTED_PENDING_VERIFICATION**, **VERIFIED**, **NON_COMPLIANT**, or **INCOMPLETE**. Supplementary Review Status shall not replace or modify Audit-Step Execution Status, LLM-Level Status, Gate State, or Audit Status;
- *Attempt Type*: **LLM_ANALYSIS** or **NOTEBOOK_EXECUTION**. An **LLM_ANALYSIS** attempt shall cite an AAI and its owning LLM Level; a **NOTEBOOK_EXECUTION** attempt shall cite an NEAI and its affected Audit Step identifiers;
- *Execution-Compliance Status*: **PENDING**, **COMPLIANT**, or **NON_COMPLIANT_EXECUTION**, governed by the **Execution-Compliance Resolution**;
- *Finding Adjudication Status*: **PROVISIONAL**, **CONFIRMED**, **REJECTED**, or **UNRESOLVED**. Every generated finding shall begin as **PROVISIONAL**. **CONFIRMED** and **REJECTED** are terminal verification states. **UNRESOLVED** is terminal for workflow accounting but shall prevent **AUDIT_COMPLETE**. Only **CONFIRMED** findings shall enter the final finding set or contribute to a risk score or gate decision; **REJECTED** and **UNRESOLVED** records shall remain in the audit trail and shall not contribute to scoring or gate decisions;

- 502 • *Performance-Claim Validity Status*: PENDING, VALID, INVALIDATED, or NOT_APPLICABLE. A
503 claim may receive VALID only after its supporting data-integrity and methodological controls
504 have been completed and verified; and
 - 505 • *Audit Status*:
 - 506 – AUDIT_INCOMPLETE: one or more required in-scope Audit Steps are PENDING, unresolved,
507 gated, unavailable, non-compliant, or otherwise not completed and verified;
 - 508 – LIMITED_SCOPE: no condition for AUDIT_INCOMPLETE exists, every remaining applicable
509 Audit Step is completed and verified, and one or more controls were formally excluded
510 as OUT_OF_SCOPE; or
 - 511 – AUDIT_COMPLETE: no condition for AUDIT_INCOMPLETE exists, every applicable Audit
512 Step is completed and verified, and no control was formally excluded.
- 513 Audit Status shall be assigned in the following precedence order: AUDIT_INCOMPLETE, then
514 LIMITED_SCOPE, then AUDIT_COMPLETE. Exactly one Audit Status shall be assigned at final-
515 ization;

516 Gate State Transition Contract

517 The Audit Manifest shall initialize the Gate State as INITIAL. A Gate State transition shall occur
518 only through one of the following verified Gate Events:

519 INITIAL + PASS1_LLM_EXECUTED -> PAUSE: Emitted when a Pass 1 LLM level completes execu-
520 tion and awaits verification.

521 PAUSE + PASS1_LEVEL_VERIFIED_CONTINUE -> INITIAL: Emitted by the authorized verifier
522 when the Pass 1 LLM level is verified, no stop condition applies, and one or more required
523 Pass 1 Audit Steps remain to be executed.

524 INITIAL|PAUSE + PASS1_VERIFIED_COMPLETE -> PROCEED: Emitted by the authorized verifier
525 after every required Pass 1 Audit Step is completed and verified and no unresolved safety
526 risk exists.

527 INITIAL|PAUSE + UNRESOLVED_SAFETY_RISK -> STOP: Emitted by the verified mandatory safety
528 pre-screen with Stop Reason UNRESOLVED_SAFETY_RISK.

529 PROCEED + LLM_LEVEL_EXECUTED -> PAUSE: Emitted after an LLM level in Passes 2–6 completes
530 execution and awaits verification.

531 PAUSE + LEVEL_VERIFIED_PROCEED -> PROCEED: Emitted by the authorized verifier when the
532 executed LLM level and its evidence are verified and no stop condition applies.

533 PAUSE + LEVEL_VERIFIED_STOP -> STOP: Emitted by the authorized verifier when verification
534 establishes a Stop Reason.

535 PROCEED|PAUSE + DETERMINISTIC_STOP_VERIFIED -> STOP: Emitted by an authorized deter-
536 ministic gate with its verified Stop Reason.

537 PROCEED|PAUSE + NON_COMPLIANT_ATTEMPT_VERIFIED -> PAUSE: Emitted when an analysis or
538 notebook execution attempt is verified as NON_COMPLIANT_EXECUTION. While this pause
539 remains active, only a compliant replacement attempt governed by the **Execution-**
540 **Compliance Resolution** may execute.

541 PAUSE + COMPLIANT_REEXECUTION_EXECUTED -> PAUSE: Emitted when an authorized compliant
542 replacement analysis or notebook execution attempt completes and awaits verification.

543 **PAUSE + COMPLIANT_REEXECUTION_VERIFIED -> PROCEED:** Emitted after the compliant govern-
544 ing replacement attempt and its required evidence have been verified and no Stop Reason
545 applies.

546 **PAUSE + COMPLIANT_REEXECUTION_VERIFIED_STOP -> STOP:** Emitted after the compliant gov-
547 erning replacement attempt is verified and its evidence establishes a Stop Reason.

548 **STOP + PROTOCOL_OVERRIDE_INVOKED -> PROCEED:** Emitted by an authorized Protocol Override
549 that covers the active Stop Reason and an LLM-level-closed set of Audit Steps to be resumed.
550 A covered set is LLM-level-closed only when, for every LLM level containing a covered Audit
551 Step, it also contains every non-completed **IN_SCOPE** Audit Step covered by that LLM level.
552 An override request whose covered set is not LLM-level-closed is a prohibited transition re-
553 quest. Each covered Audit Step whose Audit-Step Execution Status is **NOT_EXECUTED_GATED**
554 shall transition to **PENDING**, and each LLM level containing the closed covered set shall tran-
555 sition from **NOT_EXECUTED_GATED** to **PENDING**, as part of the same atomic Gate transition.
556 Audit Steps and LLM levels not covered by the override shall remain **NOT_EXECUTED_GATED**.

557 **STOP + SUPPLEMENTARY_REVIEW_AUTHORIZED -> STOP:** Emitted when supplementary analysis is
558 authorized without resuming the sequential audit protocol.

559 **STOP + SUPPLEMENTARY_LLM_EXECUTED -> STOP:** Emitted when an authorized supplementary
560 LLM level completes execution. Its Supplementary Review Status shall become
561 **EXECUTED_PENDING_VERIFICATION**.

562 **STOP + SUPPLEMENTARY_LLM_VERIFIED -> STOP:** Emitted after the supplementary LLM output
563 and evidence are verified. Its Supplementary Review Status shall become **VERIFIED**.

564 **STOP + SUPPLEMENTARY_LLM_NON_COMPLIANT -> STOP:** Emitted when supplementary LLM exe-
565 cution is non-compliant. Its Supplementary Review Status shall become **NON_COMPLIANT**,
566 and its outputs shall be excluded from supplementary evidence.

567 While **INITIAL**, only audit-package registration, scope registration, the mandatory safety pre-
568 screen, and remaining Pass 1 controls may execute. While **PAUSE**, no new Audit Step or LLM
569 level may begin except the compliant replacement attempt expressly permitted above. Verifica-
570 tion, adjudication, transition processing, and evidence recording for the activity that caused the
571 pause remain authorized. While **STOP**, no audit activity may execute except an authorized Pro-
572 tocol Override transition or a Supplementary Review Authorization that leaves the Gate State
573 unchanged.

574 Every transition record shall contain the prior Gate State, Gate Event, next Gate State, au-
575 thorizing reviewer or deterministic control, effective Audit Step identifiers, timestamp, evidence
576 references, and Stop Reason when the next state is **STOP**. A **STOP** record shall also identify every
577 Audit Step consequently assigned Audit-Step Execution Status **NOT_EXECUTED_GATED**.

578 Every transition to **STOP** shall atomically assign Audit-Step Execution Status
579 **NOT_EXECUTED_GATED** to every non-completed **IN_SCOPE** Audit Step whose execution is pro-
580 hibited by that stop. Each pending or unexecuted sequential-protocol LLM level for which all
581 non-completed **IN_SCOPE** covered Audit Steps are so gated shall atomically receive LLM-Level
582 Status **NOT_EXECUTED_GATED**. A transition to **STOP** is invalid unless these assignments and their
583 identifiers are included in the same transition record. A supplementary LLM level created under
584 a Supplementary Review Authorization is not a sequential-protocol LLM level and is governed
585 by Supplementary Review Status instead of this automatic gating assignment.

586 Any state-and-event pair not enumerated above is prohibited. A prohibited transition request shall
587 be rejected, shall leave the Gate State unchanged, and shall create a Gate Transition Violation

588 record with Status `PENDING_ADJUDICATION`. The Audit Status shall be `AUDIT_INCOMPLETE` while
589 any such record is `PENDING_ADJUDICATION` or `UNRESOLVED`.

590 An authorized human reviewer shall adjudicate each Gate Transition Violation as exactly one of:

- 591 • `REJECTED_NO_EFFECT`: the prohibited request produced no state or external effect and the
592 violation record is terminal;
- 593 • `CORRECTED_BY_VALID_EVENT`: the prohibited request produced no state or external effect,
594 a separately recorded valid Gate Event was processed, and the violation record links that
595 transition and is terminal; or
- 596 • `UNRESOLVED`: the absence of unauthorized effects or a valid correction cannot be verified; the
597 record is terminal for accounting and continues to require `AUDIT_INCOMPLETE`.

598 The Gate Transition Violation record shall contain the attempted prior state, event, requested
599 next state, requester, timestamp, affected Audit Steps, observed effects, adjudicator, evidence,
600 and final violation status.

601 **Execution-Compliance Resolution**

602 This **Execution-Compliance Resolution** governs every sequential-protocol `LLM_ANALYSIS`
603 attempt and every `NOTEBOOK_EXECUTION` attempt. It does not govern a supplementary
604 `LLM_ANALYSIS` attempt created under a Supplementary Review Authorization; such an attempt
605 is governed exclusively by the supplementary noncompliance rule in **Protocol Overrides and**
606 **Supplementary Reviews**.

607 When an attempt governed by this Resolution is recorded as `NON_COMPLIANT_EXECUTION`, its out-
608 puts shall be excluded from finding adjudication, scoring, gate decisions, pass claims, and evidence
609 of control completion. For an `LLM_ANALYSIS` attempt, the owning LLM Level shall transition to
610 `PENDING_COMPLIANT_REEXECUTION`. For a `NOTEBOOK_EXECUTION` attempt, each affected Audit Step
611 shall transition to Audit-Step Execution Status `PENDING_COMPLIANT_REEXECUTION`; no LLM-Level
612 Status transition is required unless an LLM Level owns a separate non-compliant analysis attempt.
613 In both cases, each affected Audit-Step Scope Classification shall remain unchanged, and no af-
614 fected Audit Step may transition to `COMPLETED`. Until a compliant replacement attempt is verified,
615 the Audit Status shall be `AUDIT_INCOMPLETE`. Verification of the non-compliant attempt shall emit
616 Gate Event `NON_COMPLIANT_ATTEMPT_VERIFIED`.

617 A replacement shall use a new identifier of the same attempt type: a new AAI for `LLM_ANALYSIS`
618 or a new NEAI for `NOTEBOOK_EXECUTION`. The non-compliant attempt shall remain in the Au-
619 dit Manifest. If the replacement is `COMPLIANT`, it shall become the governing attempt for its
620 owner and affected Audit Steps. Completion of the replacement execution shall emit Gate Event
621 `COMPLIANT_REEXECUTION_EXECUTED`. For an `LLM_ANALYSIS` replacement, the owning LLM Level
622 shall transition to `PAUSED_FOR_VERIFICATION` and may transition to `VERIFIED` only after out-
623 put and evidence verification. For a `NOTEBOOK_EXECUTION` replacement, each affected Audit Step
624 shall remain `PENDING_COMPLIANT_REEXECUTION` until its execution evidence is verified. In either
625 case, an affected Audit Step may transition to `COMPLETED` only after verification of the compliant
626 governing attempt.

627 For a replacement owned by a Pass 1 LLM Level, that verification shall emit
628 `PASS1_LEVEL_VERIFIED_CONTINUE` when required Pass 1 Audit Steps remain,
629 `PASS1_VERIFIED_COMPLETE` when all required Pass 1 Audit Steps are completed and veri-
630 fied, or `LEVEL_VERIFIED_STOP` when a verified Stop Reason applies. For a replacement in
631 Passes 2–6, verification shall emit `COMPLIANT_REEXECUTION_VERIFIED` when no Stop Reason
632 applies or `COMPLIANT_REEXECUTION_VERIFIED_STOP` when a verified Stop Reason applies.

633 Historical `NON_COMPLIANT_EXECUTION` records shall not independently prevent `AUDIT_COMPLETE`
634 after every affected in-scope control has been completed and verified through a compliant govern-
635 ing attempt. If no compliant governing attempt is verified, the final Audit Status shall remain
636 `AUDIT_INCOMPLETE`.

637 **Protocol Overrides and Supplementary Reviews**

638 A *Protocol Override* is a prospective authorization declared during Pass 1 scope recording for a
639 specified future Stop Reason. It shall identify the authorizing human reviewer, rationale, per-
640 mitted audit steps, applicable constraints, and expiration condition. Invocation of an applicable
641 override shall emit Gate Event `PROTOCOL_OVERRIDE_INVOKED`; the resulting transition is governed
642 exclusively by the *Gate State Transition Contract*.

643 A *Supplementary Review Authorization* is a separate authorization issued after a verified `STOP`. It
644 shall emit Gate Event `SUPPLEMENTARY_REVIEW_AUTHORIZED` and shall not change the Gate State,
645 resume the sequential audit protocol, alter any Audit-Step Execution Status `NOT_EXECUTED_GATED`,
646 satisfy an unexecuted audit step, or support an unrestricted audit status. Its outputs shall be
647 identified as supplementary evidence and recorded separately from the executed in-scope protocol
648 results.

649 Each authorization shall receive a unique Supplementary Review Identifier, identify its authoriz-
650 ing human reviewer, permitted question, source snapshot, referenced Audit Steps, allowed tools,
651 expiration condition, and supplementary LLM level when applicable, and initialize Supplementary
652 Review Status as `AUTHORIZED`. A supplementary LLM level shall receive its own AAI and shall
653 not be treated as execution of any gated Audit Step.

654 After an authorized supplementary LLM level executes, set its LLM-Level Status to
655 `PAUSED_FOR_VERIFICATION` and emit Gate Event `SUPPLEMENTARY_LLM_EXECUTED`. An authorized
656 human reviewer or approved deterministic tool shall then assign Execution-Compliance Status. A
657 compliant supplementary level whose outputs and evidence are verified shall transition to LLM-
658 Level Status `VERIFIED` and emit `SUPPLEMENTARY_LLM_VERIFIED`. A non-compliant supplementary
659 level shall transition to LLM-Level Status `VERIFIED` as a verified non-compliance determina-
660 tion and emit `SUPPLEMENTARY_LLM_NON_COMPLIANT`; its outputs shall not enter supplementary
661 evidence. This supplementary noncompliance transition is the exclusive resolution of a supple-
662 mentary LLM_ANALYSIS attempt: it shall not assign `PENDING_COMPLIANT_REEXECUTION`, shall not
663 emit `NON_COMPLIANT_ATTEMPT_VERIFIED`, and shall not authorize a replacement attempt under
664 the **Execution-Compliance Resolution**. A further supplementary attempt requires a new Sup-
665plementary Review Authorization and a new supplementary LLM level. An authorization that
666 expires without a verified compliant result shall receive Supplementary Review Status `INCOMPLETE`.
667 None of these events shall change Gate State from `STOP`.

668 **Approved Execution Runner and Containment Contract**

669 An LLM shall not claim that it constructed an environment, executed project code, or verified
670 runtime behavior unless those actions were performed through a human-authorized or approved
671 deterministic runner.

672 The runner shall return an evidence record containing:

- 673 • the execution command or workflow invocation;
- 674 • the audited source, package, and environment identities;
- 675 • the containment configuration;
- 676 • the execution start and completion states;

677 • the exit status and execution-log references;

678 • the observed runtime and resource-limit outcomes; and

679 • the produced artifact manifest.

680 Execution shall occur in an isolated, non-privileged environment with:

681 • project source and declared input data mounted read-only;

682 • a dedicated writable output location;

683 • no access to host credentials, secrets, or unrelated files;

684 • network access denied unless explicitly authorized or allowlisted;

685 • applicable process, runtime, memory, and storage limits;

686 • no privileged operations or writes outside the authorized surface;

687 • no destructive modification of source data, project files, prior artifacts, or external resources;
688 and

689 • recorded substitutes, snapshots, or authorization for required external services.

690 Environment construction and notebook execution shall be separate controlled stages. Depend-
691 encies shall not be installed, upgraded, or replaced during execution unless explicitly authorized and
692 recorded in the resolved environment.

693 If the required containment or execution capability is unavailable, execution shall not proceed.
694 The audit shall remain `AUDIT_INCOMPLETE` unless the unavailable control is formally excluded, in
695 which case it shall be `LIMITED_SCOPE`.

696 **Verification Workflow**

697 *1. Mandatory non-executing safety pre-screen:*

698 Before environment construction, import, or execution, execute and record the mandatory
699 safety pre-screen exactly as defined in Pass 1. Pass 1 is the exclusive authority for its
700 dispositions, Stop Reason, Gate State, downstream step statuses, and Audit Status.

701 *2. Context-window compliance check:*

702 Verify that the declared analysis mode complies with the Context-Window Constraints.
703 A non-compliant monolithic analysis shall be recorded and resolved exclusively under the
704 **Execution-Compliance Resolution**. Compliant analysis shall use the chunked or tool-
705 assisted mode required by the Context-Window Constraints.

706 *3. Triage and controlled execution:*

707 After the safety pre-screen has cleared, complete and verify the remaining non-executing
708 Pass 1 controls. Pass 1 shall not construct the execution environment or execute project
709 code.

710 After Pass 1 verification emits Gate Event `PASS1_VERIFIED_COMPLETE` and the Gate State
711 Transition Contract sets the Gate State to `PROCEED`, Pass 2 shall construct the approved
712 isolated environment and perform the required fresh-kernel, end-to-end execution under the
713 Approved Execution Runner and Containment Contract.

714 LLM levels within Passes 2–6 shall execute only when the Gate State is `PROCEED`, except
715 for the compliant replacement attempt expressly permitted while `PAUSE` by the Gate State

Transition Contract. All post-STOP activity is governed exclusively by **Protocol Overrides and Supplementary Reviews**.

4. *Extraction-record promotion:*

After extraction, apply the *Extraction-Record and Provisional-Finding Promotion Contract*. Record every promotion link, Finding Key, Provisional Finding Identifier, affected Audit Step identifiers, and unpromoted Extraction Record in the Audit Manifest. Only promoted objects with Finding Adjudication Status PROVISIONAL may enter finding de-duplication and evidence verification.

5. *De-duplication:*

Apply the *Finding Identity and De-duplication Contract* to every provisional finding. Record the canonical Finding Key, surviving Provisional Finding Identifier, duplicate references, and aggregated evidence in the Audit Manifest.

6. *Evidence verification:*

After each executed LLM level, set the LLM-Level Status to PAUSED_FOR_VERIFICATION. An authorized supplementary LLM level executed under STOP shall emit Gate Event SUPPLEMENTARY_LLM_EXECUTED. A non-replacement Pass 1 level shall emit Gate Event PASS1_LLM_EXECUTED; a non-replacement sequential level in Passes 2–6 shall emit Gate Event LLM_LEVEL_EXECUTED. A compliant replacement level shall instead emit COMPLIANT_REEXECUTION_EXECUTED under the **Execution-Compliance Resolution**. The Gate State Transition Contract shall apply the event-specific transition.

An authorized human reviewer or approved deterministic tool shall verify the outputs and evidence. A compliant supplementary LLM level shall transition to LLM-Level Status VERIFIED and emit SUPPLEMENTARY_LLM_VERIFIED; a non-compliant supplementary level shall transition to VERIFIED as a verified non-compliance determination and emit SUPPLEMENTARY_LLM_NON_COMPLIANT. After a non-supplementary LLM-Level Status transitions to VERIFIED, event selection for a compliant replacement level shall be governed exclusively by the **Execution-Compliance Resolution**. For a non-replacement Pass 1 level, the verifier shall emit PASS1_LEVEL_VERIFIED_CONTINUE when required Pass 1 Audit Steps remain, PASS1_VERIFIED_COMPLETE when all required Pass 1 Audit Steps are completed and verified, or LEVEL_VERIFIED_STOP when a verified Stop Reason applies. For a non-replacement level in Passes 2–6, the verifier shall emit LEVEL_VERIFIED_PROCEED when no Stop Reason applies or LEVEL_VERIFIED_STOP when a verified Stop Reason applies. The Gate State Transition Contract shall determine the resulting Gate State. A level recorded as NOT_EXECUTED_GATED shall not initiate a verification pause.

Verify every provisional finding, severity recommendation, applicability classification, checklist status, and pass claim. A pass claim requires affirmative evidence for the assessed control.

7. *Adjudication:*

Assign each provisional finding exactly one Finding Adjudication Status of CONFIRMED, REJECTED, or UNRESOLVED. Record every Audit-Step Scope Classification, Audit-Step Execution Status, and LLM-Level Status separately. A gated item is terminal for workflow accounting but is not considered completed or satisfied.

Final findings, severities, risk scores, invalidations, gate decisions, release conclusions, and audit status may enter the Final Audit Manifest only after all executed in-scope steps have reached a terminal verification state and all unexecuted steps and levels have been classified.

761 **Stable Source-Reference Protocol**

762 All audit passes and levels shall use a common source-reference map bound to the audited source
763 snapshot and its cryptographic hash.

764 Each notebook cell shall use its native `cell.id` when present. Otherwise, it shall receive a fallback
765 identifier based on its zero-based physical position in the audited snapshot, formatted as C000,
766 C001, and so forth. Identifiers shall be retained for markdown, raw, empty, failed, and executable
767 cells; excluding a cell from a report shall not renumber the remaining cells.

768 Display block numbers may be added for readability but shall not serve as authoritative references.
769 Each grouped block shall list its underlying source identifiers, for example:

770 Block 4 [Source cells: C007, C009]

771 References to project-local modules shall use the project-relative path and line range, for example:

772 src/preprocessing.py:45-78

773 Findings and verification records shall cite the applicable snapshot hash and source reference. A
774 revised source shall constitute a new snapshot with a new reference map, and revision records
775 shall link the affected references across snapshots.